

Solution : Exercices MapReduce

Configuration de l'environnement

1. Prérequis

- Docker
- Java 8+
- Maven

2. Configuration de l'environnement Docker Hadoop

Tout d'abord, créons un conteneur Docker avec Hadoop :

```
# Télécharger l'image Docker Hadoop
sudo docker pull apache/hadoop:3

# Créer un réseau pour nos conteneurs Hadoop
sudo docker network create hadoop-network

# Démarrer le conteneur Hadoop
sudo docker run -d \
  --name hadoop-master \
  --network hadoop-network \
  -p 9870:9870 \
  -p 8088:8088 \
  apache/hadoop:3
```

Exercice 1 : Analyse des ventes

Solution 1.1 : Total des ventes par ville

Créez un nouveau projet Maven avec la structure suivante :

```
sales-analysis/
├─ pom.xml
├─ src/
│   └─ main/
│       └─ java/
│           └─ com/
│               └─ bigdata/
│                   └─ SalesByCity.java
│                       └─ models/
│                           └─ Sale.java
```

Dépendances Maven (pom.xml) :

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
```

```

http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <groupId>com.bigdata</groupId>
  <artifactId>sales-analysis</artifactId>
  <version>1.0-SNAPSHOT</version>

  <properties>
    <hadoop.version>3.3.4</hadoop.version>
    <maven.compiler.source>8</maven.compiler.source>
    <maven.compiler.target>8</maven.compiler.target>
  </properties>

  <dependencies>
    <dependency>
      <groupId>org.apache.hadoop</groupId>
      <artifactId>hadoop-client</artifactId>
      <version>${hadoop.version}</version>
    </dependency>
    <dependency>
      <groupId>org.apache.hadoop</groupId>
      <artifactId>hadoop-common</artifactId>
      <version>${hadoop.version}</version>
    </dependency>
  </dependencies>
</project>

```

Sale.java

```

package com.bigdata.models;

import java.util.Date;

public class Sale {
    private Date date;
    private String city;
    private String product;
    private double price;

    // Constructeurs, getters et setters
    public Sale(Date date, String city, String product, double price) {
        this.date = date;
        this.city = city;
        this.product = product;
        this.price = price;
    }

    // Getters et setters
}

```

SalesByCity.java

```
package com.bigdata;

import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.DoubleWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

import java.io.IOException;

public class SalesByCity {
    public static class SalesMapper extends Mapper<Object, Text, Text,
DoubleWritable> {
        private Text city = new Text();
        private DoubleWritable price = new DoubleWritable();

        public void map(Object key, Text value, Context context) throws IOException,
InterruptedException {
            String[] fields = value.toString().split("\\s+");
            if (fields.length == 4) { // date city product price
                city.set(fields[1]);
                price.set(Double.parseDouble(fields[3]));
                context.write(city, price);
            }
        }
    }

    public static class SalesReducer extends Reducer<Text, DoubleWritable, Text,
DoubleWritable> {
        private DoubleWritable result = new DoubleWritable();

        public void reduce(Text key, Iterable<DoubleWritable> values, Context
context)
            throws IOException, InterruptedException {
            double sum = 0;
            for (DoubleWritable val : values) {
                sum += val.get();
            }
            result.set(sum);
            context.write(key, result);
        }
    }

    public static void main(String[] args) throws Exception {
        Configuration conf = new Configuration();
```

```

Job job = Job.getInstance(conf, "sales by city");

job.setJarByClass(SalesByCity.class);
job.setMapperClass(SalesMapper.class);
job.setReducerClass(SalesReducer.class);

job.setOutputKeyClass(Text.class);
job.setOutputValueClass(DoubleWritable.class);

FileInputFormat.addInputPath(job, new Path(args[0]));
FileOutputFormat.setOutputPath(job, new Path(args[1]));

System.exit(job.waitForCompletion(true) ? 0 : 1);
}
}

```

Exécution du Job

1. Créez un fichier de données d'exemple (ventes.txt) :

```

2024-01-15 Paris Laptop 999.99
2024-01-15 Lyon Phone 599.99
2024-01-16 Paris Tablet 299.99
2024-01-16 Marseille Laptop 1099.99
2024-01-17 Lyon Phone 649.99

```

2. Construisez le JAR :

```
mvn clean package
```

3. Copiez les fichiers dans le conteneur Hadoop :

```

sudo docker cp target/sales-analysis-1.0-SNAPSHOT.jar hadoop-master:/tmp/
sudo docker cp ventes.txt hadoop-master:/tmp/

```

4. Exécutez le job MapReduce :

```

sudo docker exec hadoop-master hadoop jar /tmp/sales-analysis-1.0-SNAPSHOT.jar
com.bigdata.SalesByCity /tmp/ventes.txt /tmp/output

```

Exercice 2 : Analyse des journaux Web

Pour le deuxième exercice, nous créerons une structure similaire mais avec une logique de mapper et de reducer différente. Voici la classe principale :

```

package com.bigdata;

import org.apache.hadoop.conf.Configuration;

```

```

import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

import java.io.IOException;
import java.util.regex.Matcher;
import java.util.regex.Pattern;

public class WebLogAnalyzer {
    public static class LogMapper extends Mapper<Object, Text, Text, IntWritable> {
        private final static IntWritable one = new IntWritable(1);
        private Text ip = new Text();
        private Pattern logPattern =
Pattern.compile("^([\\d.]+).*\"\\s(\\d{3})\\s\"");

        public void map(Object key, Text value, Context context) throws IOException,
InterruptedException {
            String line = value.toString();
            Matcher matcher = logPattern.matcher(line);

            if (matcher.find()) {
                String ipAddress = matcher.group(1);
                String statusCode = matcher.group(2);

                // Pour les requêtes totales
                ip.set(ipAddress);
                context.write(ip, one);

                // Pour les requêtes réussies (200)
                if (statusCode.equals("200")) {
                    ip.set(ipAddress + "_200");
                    context.write(ip, one);
                }
            }
        }
    }

    public static class LogReducer extends Reducer<Text, IntWritable, Text,
IntWritable> {
        private IntWritable result = new IntWritable();

        public void reduce(Text key, Iterable<IntWritable> values, Context context)
            throws IOException, InterruptedException {
            int sum = 0;
            for (IntWritable val : values) {
                sum += val.get();
            }
        }
    }
}

```

```

        result.set(sum);
        context.write(key, result);
    }
}

public static void main(String[] args) throws Exception {
    Configuration conf = new Configuration();
    Job job = Job.getInstance(conf, "web log analyzer");

    job.setJarByClass(WebLogAnalyzer.class);
    job.setMapperClass(LogMapper.class);
    job.setReducerClass(LogReducer.class);

    job.setOutputKeyClass(Text.class);
    job.setOutputValueClass(IntWritable.class);

    FileInputFormat.addInputPath(job, new Path(args[0]));
    FileOutputFormat.setOutputPath(job, new Path(args[1]));

    System.exit(job.waitForCompletion(true) ? 0 : 1);
}
}

```

Exécution de l'analyse des journaux Web

1. Créez des journaux Web d'exemple (weblogs.txt) :

```

192.168.1.1 - - [12/May/2023:15:30:45 +0000] "GET /page1 HTTP/1.1" 200 1234
192.168.1.2 - - [12/May/2023:15:31:02 +0000] "GET /page2 HTTP/1.1" 404 567
192.168.1.1 - - [12/May/2023:15:32:10 +0000] "GET /page1 HTTP/1.1" 200 789
192.168.1.3 - - [12/May/2023:15:32:35 +0000] "GET /page3 HTTP/1.1" 200 987

```

2. Exécutez le job :

```

sudo docker cp weblogs.txt hadoop-master:/tmp/
sudo docker exec hadoop-master hadoop jar /tmp/sales-analysis-1.0-SNAPSHOT.jar
com.bigdata.WebLogAnalyzer /tmp/weblogs.txt /tmp/log-output

```

La sortie affichera à la fois le nombre total de requêtes par IP et les requêtes réussies (marquées avec le suffixe _200) par IP.

Résultats des Jobs MapReduce

Résultats de l'analyse des ventes

Voici les résultats du job d'analyse des ventes :

```

Lyon      1249.98
Marseille 1099.99

```

Paris	1299.98
-------	---------

Résultats de l'analyse des journaux Web

Voici les résultats du job d'analyse des journaux Web :

192.168.1.1	2
192.168.1.1_200	2
192.168.1.2	1
192.168.1.3	1
192.168.1.3_200	1